Interfaces

Agents, the world, the parameters, and the rulings

2026

Abstract

The economy in `automation-shock.tex` and `prompt-so-far.md` is a treasury-pole gold demo: typed agents act through a gated command surface, and the thing that combines them is addition of T-accounts. This document records the containers, the serialisable world, and the knobs. Part IV records the rulings that closed the open list. The functions inside the scripted leaves are left to the agent that implements them. `plan-so-far.md` is earlier. Where it still says one bank, one education stock, withdrawable mine deposits, household fission, or debts cancelled in both directions, it is not the spec typed here.

Contents

Vocabulary **2**

I The agents **2**
- I.1 Household . . . 3
- I.2 Entrepreneur, and the tech summand . . . 4
- I.3 Mine owner . . . 5
- I.4 Government . . . 5
- I.5 Bank . . . 5
- I.6 Auditor . . . 6
- I.7 Registries . . . 6
- I.8 Gates . . . 7
- I.9 The engine . . . 8
- I.10 The month . . . 8
- I.11 Leaves behind the containers . . . 9

II The world **9**
- II.1 Finite indices . . . 9
- II.2 People and firms . . . 10
- II.3 Houses and mines . . . 11
- II.4 The financial chart . . . 11
- II.5 Markets, disclosure, archive . . . 12
- II.6 Banks, government, logs, chance . . . 13
- II.7 Invariant . . . 14

III The parameters **14**
- III.1 Fixed by the spec . . . 14
- III.2 Knobs . . . 14

IV The rulings **15**
IV.1 Two banks . . . 15
IV.2 General health . . . 16
IV.3 The incumbent . . . 16
IV.4 Discretionary purchases go last . . . 16
IV.5 An opening shortfall folds . . . 16
IV.6 Maintenance is paid to the builder . . . 16
IV.7 No time-preference field . . . 16
IV.8 The scripted leaves . . . 16

Vocabulary

A container is a pair (P, R). P is the set of shapes an agent can be asked. For each shape p, $R(p)$ is the set of positions that count as an answer. A program of that container has type $(p : P) \to R(p)$. A raw model has type String → String. The decoder is what makes the second into the first. A reply that does not decode throws. It is never rewritten into a legal-looking position.

A container morphism from (P, R) to (Q, T) is a delegate $u : P \to Q$ and an amalgamation $f : (p : P) \to T(u(p)) \to R(p)$. Control goes down through u. Data comes back up through f.

The state object on a set S is the container $\Delta S = S \triangleright (\lambda _.\, S)$. MacBeth's unfolding theorem says a morphism $\Delta S \otimes X \to Y$ is exactly a forward map, which reads the state and returns an output shape, and a backward map, which splits into a writeback of a new state and an ordinary reply. In this economy there is one such worker. Its state is the world. The agents are memoryless leaves that worker delegates to. Last year's job listings, standing raw-material orders, sales, and balance sheets are fields of the world, read back through the command surface. Copying them into an agent would fork the books.

What is not that state: the working directory, the turn counter, token budgets, and the parameters of Part III. On a scripted run the intent log is an append-only file beside the world. On a live run the harness transcript is a second register, Δ(Transcript), stamped with real ids and real times. It is not seeded. A seeded clock on a live run is a false audit.

The four combinators are not interchangeable. $\otimes$ asks every agent and owes every reply. $\times$ asks every agent and owes one reply. A market phase is a tensor: every poster and every buyer answers. It is not a product. Sequential composition $\triangleleft$ does not distribute over the sum $+$, and the TypeScript conditional types distribute it anyway. `seqC(phase, sumC(food, welfare))` silently means "decide, then ask". The month is ordinary scheduler code, in the manner of `stalin/branch.ts`, not a `seqC` over a sum.

A gate is a smart constructor. It returns Ok(e) or Refused(r). A refusal is a position, not an exception. The reducer applies an Ok effect and does not judge it again. An empty reason never reaches the reducer. The harness records `attempted` before the call, then the outcome, under one call id.

The registry is the allowlist of commands. An agent cannot name a command that is absent from its registry. From inside the agent, a missing command, a gate refusal, and a broken tool are the same position.

I The agents

Six leaves answer shapes. The engine in §I.9 is not a leaf. It has no reason and no registry. Public hospitals, public universities, private hospitals, private universities, supermarkets, nightclubs, and tech companies are firms. A firm is a bundle of books and machines. It is not a seventh agent. The government leaf operates the public firms. An entrepreneur leaf operates the firm it owns. A mine is a parcel with an owner, not a firm and not a career.

Every mutating shape carries `reason: string`. Read shapes do not. The shared answer of a mutation is `Act`.

```
type Act =
  | { tag: "done"; correlationId: string }
  | { tag: "rationed"; correlationId: string; filled: Qty }
  | { tag: "refused"; reason: string };

type Read<T> = { tag: "read"; value: T };
```

`rationed` is a committed partial fill. A lottery loss with `filled: 0` is still an outcome of a legal order. It is written to both logs. `refused` is not committed. It appears only in the intent log.

I.1 Household

Every living person is a household leaf, including entrepreneurs and mine owners. The extra shapes of those roles are summands, present only while the role holds. Losing the firm or the last mine removes the summand. That is a change of fibre, the same device as `ExportChoice` in `stalin/containers.ts`.

```
type Objective =
  | "happiness" | "health" | "education"
  | "savings" | "investment";

type HouseholdC =
  | Fib<{ tag: "books" }, Read<PersonView>>
  | Fib<{ tag: "prices" }, Read<PriceList>>
  | Fib<{ tag: "listings" }, Read<Listing[]>>
  | Fib<{ tag: "disclosure"; issuer: Id }, Read<Disclosure>>
  | Fib<{ tag: "jobArchive" }, Read<JobListing[]>>
  | Fib<{ tag: "bidEducation"; career: Career; reason: string }, Act>
  | Fib<{ tag: "bidCure"; disease: Disease; reason: string }, Act>
  | Fib<{ tag: "buy"; good: Good; qty: Qty; seller?: Id;
        reason: string }, Act>
  | Fib<{ tag: "buyCure"; disease: Disease; provider: Id;
        reason: string }, Act>
  | Fib<{ tag: "keepJob"; reason: string }, Act>
  | Fib<{ tag: "applyJob"; listing: Id; reason: string }, Act>
  | Fib<{ tag: "applyRent"; house: Id; reason: string }, Act>
  | Fib<{ tag: "offerHouse"; house: Id; ask: Money;
        reason: string }, Act>
  | Fib<{ tag: "occupyOrLet"; house: Id;
        mode: "occupy" | "let"; reason: string }, Act>
  | Fib<{ tag: "postRent"; house: Id; rent: Money;
        reason: string }, Act>
  | Fib<{ tag: "tradeMine"; mine: Id; side: "buy" | "sell";
        qty: Qty; limit: Money; reason: string }, Act>
  | Fib<{ tag: "acceptDebt"; instrument: Id; reason: string }, Act>
  | Fib<{ tag: "tradeShare"; bank: BankId; side: "buy" | "sell";
        qty: Qty; limit: Money; reason: string }, Act>
  | Fib<{ tag: "depositMine"; mine: Id; bank: BankId;
        reason: string }, Act>
  | Fib<{ tag: "requestLoan"; bank: BankId; amount: Money;
        collateral: Id; reason: string }, Act>;
```

`Good` is `staple`, `luxuryFood`, `nightclub`, `tuition`. Luxury food is the only one that raises the health stock. `tuition` names a `Career`. A private cure is not a `Good`. It is `buyCure`: ten units of one specialty, from a named hospital. It removes that disease and adds nothing to the stock. There is no clinical good and no free health grant. Names used as positions and not defined in this section (`PersonView`, `PriceList`, `Rate`, `FlowRow`, and the rest) are projections of the world types in Part II. They are not a second chart.

`buy` may name a seller. If it does not, the engine fills from the cheapest posted lots. A glut does not need loyalty. A seller who posts more units than buyers ask is left with inventory, and next month's price falls.

`keepJob` and `applyJob` exist only on the job step of the special day. Health does not remove either shape. An incumbent who wants the post keeps it at the same salary while unhealthy, until the firm fires them. `applyJob` exists only when the person's level in that career is at least the listing's generation. Among non-incumbents the healthiest eligible applicant is hired. A person at low health, including a hard zero, is hired only when eligible applicants with higher health do not fill the slots. Ties at one health are a seeded lottery. `offerHouse`, `occupyOrLet`, `postRent`, and `applyRent` exist only on their special-day steps. `tradeMine` of a liened or deposited mine cannot be written. `depositMine` cannot be written on a liened mine. A lien blocks transfer, and a deposit is a transfer.

`acceptDebt` exists only when the sum of the person's deposits at the two banks is at least the savings threshold. The lender reads `disclosure` and either accepts or does not answer that instrument. Silence is not a shape. The issuer ratchets the rate next month because the offer went unfilled.

Reproduction has no shape. The engine applies the gate after the month's trades. A parent in the voluntary band reproduces according to the probability for its `Objective` (Part III). A parent past the mandatory ceiling reproduces with probability one.

I.2 Entrepreneur, and the tech summand

```
type FirmC =
  | Fib<{ tag: "postPrice"; firm: Id; good: Good; price: Money;
        qty: Qty; reason: string }, Act>
  | Fib<{ tag: "postCure"; firm: Id; disease: Disease;
        price: Money; qty: Qty; reason: string }, Act>
  | Fib<{ tag: "postJob"; firm: Id; career: Career;
        generation: number; salary: Money; slots: number;
        reason: string }, Act>
  | Fib<{ tag: "assign"; firm: Id; person: Id;
        specialty: Specialty | null; reason: string }, Act>
  | Fib<{ tag: "fire"; firm: Id; person: Id; reason: string }, Act>
  | Fib<{ tag: "orderMachine"; firm: Id; machine: MachineType;
        generation: number; qty: Qty; limit: Money;
        reason: string }, Act>
  | Fib<{ tag: "upgradeOwn"; firm: Id; machine: Id;
        mode: "fire" | "redeploy"; reason: string }, Act>
  | Fib<{ tag: "issueDebt"; firm: Id; principal: Money;
        installment: Money; months: number; reason: string }, Act>;

type TechC =
  | Fib<{ tag: "strategy"; firm: Id;
        mode: "stay" | "expand" | "upgrade";
        reason: string }, Act>
  | Fib<{ tag: "orderMaterial"; firm: Id; material: Material;
        qty: Qty; limit: Money; rollover: boolean;
        reason: string }, Act>;
```

`postCure` passes the gate only for a firm of kind `privateHospital`. A public hospital does not sell cures. `assign` is how a generic teacher or doctor is pointed at a subject or a specialty for the coming month. `Specialty` is the five diseases. It is not a career. `upgradeOwn` exists only on the job step, and only for a machine the firm already holds. `orderMachine` exists only on that same step. `fire` exists in any month.

The tech summand is present only for a firm of kind `tech`. `strategy` is the product choice. `stay` posts the current generation at the current volume. `expand` is a plan to buy more machines of that generation and list more engineer slots. `upgrade` is a plan to list an engineer slot at

the next level and to offer the new generation. The plan does not itself mint a generation. A generation enters the world when a built machine of that generation is delivered.

`orderMaterial` with `rollover: true` writes a standing order into the world. The engine resubmits it each month at a higher limit, up to the recorded ceiling, until it fills or the ceiling binds.

An entrepreneur is the household container plus `FirmC` scoped to the owned firm, plus `TechC` when the firm is a tech firm. One person owns at most one firm.

I.3 Mine owner

A mine owner is a household with at least one undeposited parcel. The mine shapes are:

```
type MineC =
  | Fib<{ tag: "postMaterial"; mine: Id; price: Money;
        maxUnits: Qty; reason: string }, Act>
  | Fib<{ tag: "sellMine"; mine: Id; ask: Money;
        reason: string }, Act>;
```

`maxUnits` cannot exceed remaining stock. Posting less than the stock is withholding. It is legal. A deposited mine is operated by the bank leaf, and these two shapes disappear for that parcel.

The indigenous are mine owners at month zero. They are not a container.

I.4 Government

```
type GovernmentC =
  | Fib<{ tag: "books" }, Read<PublicBooks>>
  | Fib<{ tag: "setTeacherMix";
        assign: { person: Id; career: Career }[];
        reason: string }, Act>
  | Fib<{ tag: "setDoctorMix";
        assign: { person: Id; disease: Disease }[];
        reason: string }, Act>
  | FirmC;
```

`FirmC` here is scoped to the public hospital and the public university. The government leaf has no `requestLoan`. It does not own collateral, and a bank loan is the secured channel. It borrows by `issueDebt`, whose disclosure is tax revenue and the salary bill. It has no shape that pays cash, transfers a house, or buys food for a citizen.

Public output is not divided across the population. Citizens `bidEducation` and `bidCure`. The engine serves homeless, then renters, then owners, and lotteries inside a tier. A winner receives one full level, or ten units of the matching specialty. A loser receives nothing. The sentence in the government section of `automation-shock.tex` that still says each citizen receives public production divided by population is the earlier rule.

I.5 Bank

The bank leaf is memoryless. It does not forecast. It has no model of the economy. The loan decision is not one of its shapes. `requestLoan` is judged by the gate in §I.8.

```
type BankC =
  | Fib<{ tag: "postLoanRate"; bank: BankId; rate: Rate;
        reason: string }, Act>
  | Fib<{ tag: "postMaterial"; bank: BankId; mine: Id;
        price: Money; maxUnits: Qty; reason: string }, Act>
  | Fib<{ tag: "sellCapital"; bank: BankId; mine: Id;
        ask: Money; reason: string }, Act>;
```

`postMaterial` exists only for mines deposited at that bank. `sellCapital` exists only for those mines. The gate allows it only when that bank's equity is negative, or its loans exceed `leverageMultiple` times its marked mine base. The quantity sold is the mine. Mines are parcels, not slices. Depositors do not withdraw. They sell shares with `tradeShare`.

`BankId` has two inhabitants, `"a"` and `"b"`. Each has its own loan rate, ore book, shares, dividend, and ten-times ceiling.

I.6 Auditor

```
type AuditorC =
  | Fib<{ tag: "invariant" }, Read<InvariantReport>>
  | Fib<{ tag: "reconcile" }, Read<ReconcileReport>>;
```

The auditor reads. Its registry has no mutation. `InvariantReport` is the trial of Part II: each financial instrument sums to zero, and each flow column of the month sums to zero. `ReconcileReport` joins the journal to the intent log on the correlation id. An alarm is a committed mutation with no recorded reason. A blocked entry is a refusal, visible only in the intent log. This join is the amalgamation. Sub-balances sum. Debits equal credits. Nothing in the economy is a semilattice of partial maps, and nothing is a picture concatenated from two reports.

I.7 Registries

Leaf	Mutations the registry contains
Household	bids, `buy`, job and rent applications, house offers, mine trades, `acceptDebt`, share trades, `depositMine`, `requestLoan`
Entrepreneur	household, plus firm prices, jobs, assignment, fire, machines, `upgradeOwn`, `issueDebt`
Tech	entrepreneur, plus `strategy` and `orderMaterial`
Mine owner	household, plus `postMaterial` and `sellMine`
Government	mixes, and firm commands on the two public firms, including `issueDebt`
Bank	`postLoanRate`, `postMaterial`, `sellCapital`
Auditor	none

No leaf has a shell. No leaf writes its own reason into the journal. The command surface writes the effect. The harness writes the reason.

I.8 Gates

Shape	Ok only when
`requestLoan`	collateral is a house or a mine, it is unlien'd, and value×LTV ≥ amount, and the bank's outstanding loans plus the amount are within `leverageMultiple` times the marked mine base
`issueDebt`	the installment times the months is an integer, at least the principal, and both are positive
`acceptDebt`	the buyer names an open offer and can pay the posted price
`buy`	the good is posted and the quantity is positive
`applyRent`	the applicant can pay the posted year, else the position is `refused` and, if no dwelling remains, health is pinned
`keepJob`	the person is the incumbent
`fire`	the person is employed at that firm
`postCure`	the firm is a private hospital
`sellCapital`	that bank's equity is negative, or its loans exceed `leverageMultiple` times its marked mine base
any mutation	`reason` is non-empty; checked before the command runs

The reducer accepts an effect, not a shape.

```
type Effect =
  | { tag: "pay"; from: Id; to: Id; amount: Money; row: FlowRow }
  | { tag: "lendBank"; bank: BankId; borrower: Id; amount: Money;
      collateral: Id; installment: Money; months: number }
  | { tag: "repayBank"; loan: Id; amount: Money }
  | { tag: "issuePrivate"; issuer: Id; holder: Id;
      principal: Money; installment: Money; months: number }
  | { tag: "transferPrivate"; instrument: Id; from: Id; to: Id;
      price: Money }
  | { tag: "writeOff"; instrument: Id }
  | { tag: "post"; listing: Listing }
  | { tag: "fill"; listing: Id; buyer: Id; qty: Qty }
  | { tag: "hire"; listing: Id; person: Id }
  | { tag: "fire"; firm: Id; person: Id }
  | { tag: "assign"; person: Id; specialty: Specialty | Career | null }
  | { tag: "deliverMachine"; firm: Id; machine: Machine }
  | { tag: "extract"; mine: Id; qty: Qty }
  | { tag: "depositMine"; mine: Id; bank: BankId; shares: Qty }
  | { tag: "seize"; loan: Id }
  | { tag: "escheat"; person: Id }
  | { tag: "grantHouse"; house: Id; person: Id }
  | { tag: "birth"; parent: Id; child: Id }
  | { tag: "death"; person: Id }
  | { tag: "cure"; person: Id; disease: Disease }
  | { tag: "levelUp"; person: Id; career: Career }
  | { tag: "pinHealth"; person: Id; cause: "homeless" | "disease" }
  | { tag: "draw"; firm: Id; owner: Id; amount: Money }
  | { tag: "dividend"; bank: BankId }
  | { tag: "standing"; order: StandingOrder };
```

`lendBank` creates a deposit owed by the lending bank. `repayBank` destroys that deposit and retires principal. Interest paid to a bank is a `pay` that lowers the payer's deposit and raises that bank's equity. It does not retire principal, and it does shrink the stock of customer deposits until the dividend pays it back out. A private loan moves an existing deposit. Writing one off removes the asset and the liability together and moves no deposit. Maintenance is a `pay` whose row is `maintenance`, from the firm that holds the machine to the tech firm in `Machine.builder`, at the contractual price for that career and generation. It runs in the opening. If the builder has folded, the charge that month is zero.

I.9 The engine

The engine is the writeback of the world worker. It runs between leaves, never as a leaf.

- Contractual flows at the opening: wages, bank installments, private installments, tax collected on last month's fills, and maintenance paid to each machine's builder.
- Disease draws. A new disease pins health at zero.
- Mark to market. A margin call sells the borrower's liquid assets in order: the two deposits, larger balance first, then bank shares, then private debt, then undeposited mine land. Foreclosure runs only if that sale is short.
- The public lottery, after the bids.
- Market clearing: cheapest lots if the buyer named no seller, then a seeded lottery on a sell-out.
- Starvation. If the food fill is zero, sell liquid assets below the posted price by the distress discount and try the food book again. Three consecutive months with a zero fill and nothing liquid left is `death`.
- Fold, when a firm cannot meet the opening flows. The waterfall is wages, then tax, then the bank, then unsecured holders. A food reserve for the months until the next special day is kept out of the estate. Unmortgaged houses are not seized.
- Birth gifts and death routing, after trade.
- The dividend, on the special day, after shares have traded.
- The invariant assertion, after every effect.

A fold at the opening cannot be prepaid by a loan issued later the same month. Obligations, then borrowing, then investment. A same-month loan that covered an opening shortfall would remove that squeeze, so the fold stays in this phase.

I.10 The month

Sellers post before buyers. Food and rent settle before discretionary purchases. Discretionary purchases are luxury food, the nightclub, tuition, and private cures. On an ordinary month the scheduler is:

1. Engine: contracts, disease, margin calls.
2. Government: the two mixes, seeing current disease counts and the job archive.
3. Tensor of households: the two bids. Engine: the lottery.
4. Tensor of firms, mine owners, and banks: prices and quantities.
5. Tensor of households: `buy` of `staple`. Engine: clear, then the starvation rule.
6. Tensor of households: discretionary buys. Engine: clear.
7. Tensor of issuers and of households: debt, mine trades, share trades, `requestLoan`, `orderMaterial`. Engine: clear, then `draw` of any firm deposit that remains.
8. Engine: births and deaths. Auditor: invariant and reconcile.

Firing is a shape in steps 2 and 7, and on the job step below. It is legal in any month. Hiring is not.

On the special-day month the same list runs, with one insertion. After staple food, and before discretionary buys, the special day runs in this order. Share trades do not run again in step 7 of that month. Mine trades and debt still do, so the dividend can be spent on them.

1. Bank shares. Buyers and sellers both answer.

2. Engine: the dividend to the holders of record just produced.
3. House sales. Winners are a seeded lottery, not a price auction.
4. Tensor of owners: `occupyOrLet`.
5. Job listings and machine orders, then `keepJob` and `applyJob`. The incumbent who wants the post keeps it. Other eligible applicants are taken in descending health, and a shortfall of healthier applicants is what lets a low-health applicant in. Incumbent salary does not change.
6. Rents are posted, tenants apply, the sitting tenant is first, and the rest are ranked by salary. The year is paid in full or not at all. Health pins at zero for anyone left undwelled.

I.11 Leaves behind the containers

Each leaf is a `Runner`. The default runner is scripted and deterministic. A `--live` flag swaps in a model client behind the same decoder. The scheduler does not know which one it called. The scripted household reads its `Objective` and the price list. The scripted entrepreneur reads last month's unit sales, which are on the firm. The scripted bank reads the other posted rate, whether last month's loan book was full, and its own loss line. None of them reads a forecast of the price level.

The formulas inside those leaves are not fixed here. The agent that implements a leaf chooses them. Part IV lists the slots.

II The world

The world is one JSON document. Money is an integer number of minor units. Quantities of goods, ore, doctor-time, and teacher-time are integers. A price is money per unit, an integer. A repayment whose installment does not divide is not a value of `issueDebt`. The existing `absfc_core.py` is the invariant on a coarser chart (`deposits`, `loans`, `bonds`, as floats). It is the prototype of `check`. It is not this chart, and a float sum is not the replay.

Ids and the month counter are fields of the world on a scripted run. A live run uses `realIds` and `realClock` in the logs and does not claim byte-identical replay.

II.1 Finite indices

```
type Career =
  | "teacher" | "doctor" | "bureaucrat"
  | "retail" | "hospitality" | "engineer";

type MachineType = Career;   // the bijection, mining excluded

type Material = "common" | "advanced";

type Disease =
  | "d0" | "d1" | "d2" | "d3" | "d4";

type FirmKind =
  | "supermarket" | "luxury" | "nightclub"
  | "privateHospital" | "privateUniversity"
  | "publicHospital" | "publicUniversity"
  | "tech";

type Objective =
  | "happiness" | "health" | "education"
  | "savings" | "investment";
```

Level n in a career is what lets a person operate a machine of that career at generation n. An advanced engineer is an engineer at level at least 1. The class is empty at month zero because every engineer starts at level 0. Switching careers sets the new career's level to 0 and leaves the old level in the record. It does not transfer.

II.2 People and firms

```
type BankId = "a" | "b";

type Deposits = Record<BankId, Money>;

type Person = {
  id: Id;
  alive: boolean;
  objective: Objective;
  health: number;          // integer stock, unbounded above
  happiness: number;
  education: Record<Career, number>;
  diseases: Disease[];     // each one pins health at 0
  starvation: number;      // consecutive foodless months, 0..3
  job: { firm: Id; career: Career; salary: Money } | null;
  dwelling: Id | null;     // a house they own and occupy, or rent
  parent: Id | null;
  children: Id[];          // birth order; index 0 is the heir
  firm: Id | null;         // at most one
  deposits: Deposits;
};

type Firm = {
  id: Id;
  kind: FirmKind;
  owner: Id;               // a person, or the government id
  deposits: Deposits;
  machines: Id[];
  inventory: Record<string, Qty>;
  salesLastMonth: Qty;
  taxOwed: Money;
};

type Machine = {
  id: Id;
  type: MachineType;
  generation: number;
  owner: Id;               // the firm that holds it
  builder: Id;             // the tech firm maintenance is paid to
};
```

Health and happiness decay by the factors in Part III, then rise by fills. Health is then cut by the ceiling $H_{\max}(\text{happiness})$. While `diseases` is non-empty, or `dwelling` is null outside the special day, health is zero and a luxury fill does not change it. Luxury food is the only fill that raises health. Staple food does not. It is the only good that resets `starvation`. A cure removes one entry from `diseases` and adds nothing to the stock. Nothing in the opening, and no public grant, adds to health. The stock does not trend up on its own.

Capacity of a firm in a career is

$$\sum_{\text{machines of that career}} \text{output}(\text{gen}) \cdot \min\left(1,\ \frac{\text{operators on hand}}{\text{ops}(\text{gen})}\right).$$

Bureaucrats are the operators of admin machines. The ratio β is that requirement at generation 0, calibrated to 1. It is not a second formula. A firm short of bureaucrats runs the productive machines it can staff, and no further.

Two purses: a firm's deposits and a person's deposits. Each purse is split across the two banks. Opening obligations are paid from the firm purse. `draw` moves what remains to the owner after the capital step. The household buys food from its own purse. Cash left inside the firm is not food. There is no `timePreference` field. Career choice reads the job archive. The five objectives already separate a patient agent from an impatient one.

II.3 Houses and mines

```
type House = {
  id: Id;
  units: number;           // positive multiple of 10
  owner: Id;
  occupant: Id | null;     // owner, tenant, or empty
  lien: Id | null;         // a bank loan id
};

type Mine = {
  id: Id;
  material: Material;
  stock: Qty;
  owner: Id;               // a person, or a bank id once deposited
  depositedAt: BankId | null;
  lien: Id | null;
};
```

A house trades only on the special day. A mine trades any month, unless deposited or liened. Depositing mints shares and sets `owner` to the bank. The parcel does not come back. Extraction subtracts from `stock` and cannot pass zero. The bank's lending base is

$$\sum_{\text{deposited mines}} \text{stock}(m) \cdot \text{ref}(\text{material}(m)),$$

where ref is a smoothed price from recent sales of undeposited mines of that material. The bank's own posted ore price is not an input. If no independent sale of that material sits inside the window, the last reference is carried forward. The optional revenue floor of Part III is off unless a run shows a hoarded mine marking high.

II.4 The financial chart

Every financial row sums to zero across holders. Real rows do not.

Instrument	Asset of	Liability of	Transfer
Monetary deposit	holder	the issuing bank	`pay`
Bank loan	the bank	the borrower	never
Private loan	current holder	the issuer	`transferPrivate`
Bank share	the holder	that bank	the share market
Interbank claim	one bank	the other bank	settlement inside `pay`

A bank loan's fields are the collateral id, the remaining principal, the installment, the months left, and the bank. A private loan's fields are the issuer, the holder, the same schedule, and the issuer kind (`firm` or `government`). There is no bond type beside that. There is no equity in a firm. The owner's claim is `draw`.

A person's liquid assets, in the order the starvation rule sells them, are the two deposits (larger balance first), bank shares, private loans held, and undeposited mines. Houses are absent from that list.

A `pay` debits the payer's largest balance first and credits the payee at that same bank. That bank's deposit liability is unchanged. `World.interbank` moves only when the payee is a bank

and the payer's balance at that bank is short. The engine debits the payer at the other bank and books the claim. Interest then raises the creditor bank's equity. Principal repayment retires the loan. Neither step credits a new customer deposit. The claim pays no interest. It is not a loan the bank leaf decides. With that row, deposits still sum to zero across holders and both banks.

The flow rows, each of which sums to zero across payers and payees, are: wages, staple, luxury food, nightclub, tuition, private cure, rent, ore, machine purchase, maintenance, tax, bank interest, bank principal, private installment, profit draw, dividend, and the trade prices of houses, mines, debt, and shares. Capital-account rows record Δ of each instrument. Bank principal repaid destroys a deposit. Bank interest raises that bank's equity and shrinks customer deposits until the dividend pays them back out. A private installment, principal and interest both, moves a deposit and does not destroy one.

II.5 Markets, disclosure, archive

```
type Listing = {
  id: Id;
  seller: Id;
  kind: "good" | "ore" | "machine" | "job"
      | "houseSale" | "rent" | "mine" | "debt" | "share";
  price: Money;
  qty: Qty;
  payload: JobPayload | DebtOffer | MachineOffer | null;
};

type DebtOffer = {
  issuer: Id;
  principal: Money;
  installment: Money;
  months: number;
};

type Disclosure = {
  issuer: Id;
  deposits: Deposits;
  revenueLastMonth: Money;
  wageBill: Money;
  debtService: Money;
  machines: { type: MachineType; generation: number }[];
  // government issuers fill the next two and leave machines empty
  taxLastMonth: Money;
  salaryObligations: Money;
};

type StandingOrder = {
  firm: Id;
  material: Material;
  qtyRemaining: Qty;
  limit: Money;
  nextBid: Money;
};

type JobArchive = {
  year: number;
  listings: { career: Career; generation: number;
              salary: Money; slots: number }[];
};
```

`JobArchive` is replaced on the job step and kept for one year. Education bids read it. The scripted household trains toward the careers that listed the most slots.

Prices for the next month are a function of this month's listing and of how much of `qty` filled. Sold out: multiply by the raise factor. Unsold: multiply by the cut factor. A competitor posted lower for the same kind: the next price is at most that post times the undercut factor. Incumbent wages are outside this rule. They change only when the person wins a different listing.

II.6 Banks, government, logs, chance

```
type Bank = {
  id: BankId;
  loanRate: Rate;
  equity: Money;           // interest and ore sales raise it; dividend and losses lower it
  losses: Money;           // unrecovered defaults, this year
};

type Government = {
  id: Id;
  deposits: Deposits;
  taxLastMonth: Money;
  salaryBill: Money;
};

type World = {
  month: number;
  nextId: number;
  rng: RngState;
  people: Record<Id, Person>;
  firms: Record<Id, Firm>;
  houses: Record<Id, House>;
  mines: Record<Id, Mine>;
  machines: Record<Id, Machine>;
  loans: Record<Id, BankLoan | PrivateLoan>;
  shares: Record<Id, { bank: BankId; holder: Id; units: Qty }>;
  banks: Record<BankId, Bank>; // exactly the keys "a" and "b"
  interbank: Money;        // claim of "a" on "b"; negative if reversed
  government: Government;
  listings: Listing[];
  standing: StandingOrder[];
  jobArchive: JobArchive;
  flows: FlowEntry[];      // this month, cleared at the boundary
};
```

`RngState` is part of the document so a lottery is replayable. Disease, sell-outs, house-sale ties, the public lottery, unpaid wages in a fold, and childless gifts of houses all draw from it and from nowhere else.

The journal and the intent log are not fields of `World`. They are two append-only files.

```
type JournalEntry = {
  correlationId: string;
  month: number;
  effect: Effect["tag"];
  // no reason
};

type IntentRecord = {
  callId: string;
  correlationId: string | null;
  month: number;
  agent: string;
  status: "attempted" | "ok" | "rationed" | "refused" | "error";
  reason: string;
};
```

The command surface appends a journal line only inside the `commit` that has already passed the reason check and the gate. Engine effects append a journal line with a cause tag and no intent record. Reconciliation alarms only on agent commits.

II.7 Invariant

After every effect, for each of the five financial instruments, the signed sum across all holders is zero. The five are the monetary deposit, the bank loan, the private loan, the bank share, and the interbank claim. After the last effect of a month, each person, firm, bank, and the government has a flow column that sums to zero: inflows minus outflows equal the change in its net financial assets. A non-zero column is a leak. Real stocks are checked only for their own laws: house units are positive multiples of ten, mine stock is non-negative, a deposited mine has no person as owner, a liened id names a live bank loan, and a dead person holds nothing.

III The parameters

These are numbers. None of them adds a shape. Ratios are the ones that move the regime: automation against the education throughput that feeds it, leverage against the distress discount, and the tax rate against the savings objective. Sweeping a level, with the ratios fixed, rescales a run.

III.1 Fixed by the spec

Name	Value	Role
Months in a year	12	the clock
Foodless months until death	3	starvation
House quantum	10	gift, escheat, sizes
Cure size	10	doctor units, all or nothing
Fertility, voluntary	10 spare units and $3G$ liquid	per child
Fertility, mandatory	100 units and $10G$ liquid	forced birth
Heir	the first id in `children`	primogeniture
Leverage ceiling	$10\times$ the marked base	new bank loans
Careers, diseases	6 and 5	the enums

The leverage multiple is on the tunable list in the prompt as well. The interface treats 10 as the default and `leverageMultiple` as the knob.

III.2 Knobs

Knob	What it sets	Default, where one is stated
G	cash gifted to a newborn	about a year of staple food
ops(c, g)	operators of career c a generation-g machine requires	falls in g; the shock
output(c, g)	units per machine per month	non-decreasing in g
β	ops(`bureaucrat`, 0) relative to other careers at generation 0	1
recipe(g)	common and advanced ore per machine of generation g	advanced rises with g
ore stock by material	finite mine stocks at month 0	enough to reach the last generation
LTV	bank loan against collateral value	none stated
`leverageMultiple`	ceiling on new bank loans	10
sales tax	per unit sold, firm to government	none stated
fire-sale discount	bank collateral and seized mines	slight, unnamed
distress discount	starvation and gift liquidation	deeper than the fire sale
δ_h, δ_q	monthly decay of health and happiness	none stated
$H_{\max}$	ceiling of health given happiness	increasing; form unset
units per level	teacher-time for one level-up	none stated
fill per unit	health from luxury food, happiness from a nightclub and from house units	none stated
disease probability	per person per month, across the five	none stated
savings threshold	minimum deposit to `acceptDebt`	none stated
raise, cut, undercut	multiplicative reprice factors	none stated
reference window	months of independent mine sales in ref	none stated
thin-market haircut	ore price used when the window is empty	off unless needed
revenue floor k	cap on a deposited mine's mark	off
food reserve	staple units kept out of a fold	the months until the next special day
strategy weights	probability of each `Objective` at birth	a distribution, not a point
fertility weights	probability of a voluntary birth given the objective	five numbers
t_0 headcount	people, by stratum and by career	full employment, all housed
t_0 firms	counts of each `FirmKind`	4 tech, 2 of each shop, 3 nightclubs, 1 of each hospital and university
t_0 parcels	mines of each material, and their owners	all indigenous; see below
t_0 prices and wages	every listing at month 0	a fixed vector

The parcel count has a constraint the old seed line "2 mines" does not meet. ref is identified from independent sales. A single parcel of a material cannot both sit inside a bank and clear a reference market. Each material needs enough parcels, and enough owners who do not deposit, that a sale can be observed. The number is a knob. The lower bound is a requirement of the valuation rule.

$H_{\max}$ needs a form before a run. The interface uses $H_{\max}(q) = h_0 + kq$ until a different form is chosen, with h_0 and k two more knobs. Health gains from luxury food above that line are discarded.

Machine purchase prices and maintenance prices at each (c, g) are part of the t_0 price vector. The maintenance price is paid each month to `Machine.builder`.

IV The rulings

IV.1 Two banks

`BankId` is `"a"` or `"b"`. Each bank has its own loan rate, deposited mines, shares, dividend, equity, and lending ceiling. A person, a firm, and the government each hold a deposit at each

bank. A new bank loan creates its deposit as a liability of the lending bank. A payment credits the payee at the bank that was debited. `World.interbank` moves only when someone must pay one bank out of a deposit at the other. The claim pays no interest.

IV.2 General health

Luxury food is the only addition to the health stock, and it is still capped by $H_{\max}(\text{happiness})$. Doctors cure diseases. A cure clears the hard zero for that disease and does not add points. Staple food resets starvation and does not touch the stock. There is no clinical good and no free health grant, so health does not trend upward by itself.

IV.3 The incumbent

`keepJob` stays in the container while health is pinned. An unhealthy worker who wants the post keeps it, at the same salary, until the firm fires them. Firing remains a firm shape in any month. `applyJob` is gated by education only. Selection among non-incumbents is descending health. A low-health applicant, including a hard zero, is hired when healthier eligible applicants do not fill the slots.

IV.4 Discretionary purchases go last

Staple food, and on the special day the year's rent, settle before luxury food, the nightclub, tuition, and private cures. On the special day the rent step still follows the job step, because applicants are ranked by the salary just determined. Investment stays after the discretionary step.

IV.5 An opening shortfall folds

Wages, debt service, tax, and maintenance are paid before any leaf may borrow. A firm that cannot meet them folds in that phase. It does not issue debt first and pay second. That is the squeeze the entrepreneur's order describes. A loan later the same month cannot reach back and undo it.

IV.6 Maintenance is paid to the builder

Each machine records the tech firm that built it. Each month the holder pays that firm the contractual maintenance price for the machine's career and generation. The payment is a flow row, so the books still balance. If the builder has folded, the charge is zero that month rather than paid to a stand-in.

IV.7 No time-preference field

Education choice reads last year's job archive and trains toward the careers that listed slots. `Person` does not carry a separate horizon. The five objectives already make some agents spend now and others accumulate.

IV.8 The scripted leaves

The containers and the world are fixed. The function inside each scripted leaf is chosen by the agent that implements it. None of those functions reads a model of the economy. The slots are:

Leaf	Chosen by the implementer
Government	which assignment to give each public teacher and doctor, and when to fire rather than `issueDebt`
Tech entrepreneur	`stay`, `expand`, or `upgrade`, and on `upgradeOwn` whether to fire or redeploy
Entrepreneur	how much of the firm deposits to leave for `draw`, against machines, ore, and debt
Lender	which fields of `Disclosure` must cover the installment, and by what multiple
Bank	the next `loanRate` and the next ore price, from the rival's post, the fullness of the loan book, and `losses`